CS 464 - Introduction to Machine Learning
Spring 2026
Instructor: Sinem Sav
Group 19 Progress Report

Salient Object Segmentation

Yunus Günay	*yunus.gunay@ug.bilkent.edu.tr*	22203758
Daib Malik	*daib.malik@ug.bilkent.edu.tr*	22201013
Muhammad Musa Qureshi	*musa.qureshi@ug.bilkent.edu.tr*	22301064
Ali Sher Mir	*sher.mir@ug.bilkent.edu.tr*	22201310

5 April 2026

Contents

1. Introduction **3**
2. Dataset **3**
2.1 Dataset Analysis 3
2.2 Dataset Preprocessing 4
3. Coding Environment **5**
4. Details of the Trained Models **5**
4.1 CNN-Based Autoencoder 6
4.2 U-Net 6
4.3 DINO-Based Segmentation Model 6
4.4 CLIP-Based Segmentation Model 7
4.5 Hyperparameters and Their Effects 7
5. Experimental Setup **8**
6. Results **9**
6.1 CNN Results 9
6.2 U-Net Results 10
6.3 DINO Results 12
6.4 CLIP Results 13
7. Future Work **14**
8. Workload Distribution **14**
9. References **15**

1. Introduction

Salient object segmentation is the task of identifying and delineating the most visually prominent objects in an image at the pixel level. Unlike image classification, which assigns a single label to an entire image, segmentation requires the model to produce a binary decision for every pixel, determining whether it belongs to the salient foreground or the background. This makes salient object segmentation a challenging problem that requires both fine-grained spatial reasoning and high-level semantic understanding.

A key difficulty arises from the fact that saliency is not purely a low-level visual property, but rather a perceptual concept. Humans identify salient objects based on a combination of contrast, context, and semantic meaning. For example, a person in a natural landscape stands out not only due to visual differences, but also because the human visual system recognizes it as a meaningful entity. Capturing this behavior computationally requires models that learn rich representations of objects and their contextual relationships.

In this project, we investigate how the choice of visual feature extractor affects segmentation performance on the **Extended Complex Scene Saliency Dataset (ECSSD)** [1, 2]. In particular, we aim to evaluate whether pretrained vision encoders, when used as frozen feature extractors with lightweight decoders, can outperform models trained entirely from scratch on a relatively small dataset. Our hypothesis is that pretrained models, having been trained on large-scale image corpora, provide semantically rich representations that improve both performance and sample efficiency.

To study this, we implement and compare four deep learning-based approaches: (i) a CNN-based autoencoder as a baseline model, (ii) a U-Net architecture trained from scratch, (iii) a segmentation model using a frozen DINO Vision Transformer encoder with a trainable decoder, and (iv) a segmentation model using a frozen CLIP Vision Transformer encoder with a trainable decoder.

All models are trained and evaluated under consistent experimental settings, including identical data splits and evaluation metrics, to ensure a fair comparison. At this stage of the project, the training pipeline is implemented, including dataset preprocessing, model architectures, and evaluation metrics. Initial experiments have been conducted, and further analysis and extended experiments are planned in subsequent stages.

2. Dataset

2.1 Dataset Analysis

The experiments in this project are conducted on the **ECSSD** [1, 2], which contains 1000 natural images along with corresponding pixel-level ground truth masks. Unlike simpler datasets with smooth backgrounds, ECSSD includes images with structurally complex and semantically meaningful scenes, making it a more challenging and realistic benchmark for salient object segmentation. The images are collected from the internet, and the ground truth masks are manually annotated by multiple human annotators [2].

Dataset Characteristics:

Each sample in the dataset consists of an RGB image as an input, and a grayscale binary mask as ground truth. The masks encode foreground (salient object) with value **1** and background with value **0**. The dataset is organized into two separate directories for images and masks, where corresponding pairs are matched using identical filenames. This structure allows straightforward pairing during data loading.

These images are provided in JPEG format, while the masks are in PNG format. The original image resolution is approximately 267 x 400 pixels (width x height). In the preprocessing stage of the project, all images and masks are resized to a fixed resolution to ensure consistency across different models.

The dataset exhibits significant variability in visual content. Images often contain cluttered and textured backgrounds, multiple objects, and overlapping structures, along with variations in illumination, contrast, and color. Additionally, salient objects appear at different scales and positions. A key characteristic of ECSSD is that saliency is determined not only by low-level visual features but also by semantic importance.

Defined Classes and Balance:

The problem is formulated as a binary segmentation task with two classes:

- **Class 1:** Salient Foreground Object
- **Class 0:** Background

There are no multiple semantic categories. The dataset exhibits **pixel-level class imbalance**, where background pixels typically dominate the image while foreground regions occupy a smaller region. This imbalance can bias models toward predicting background regions. Therefore, evaluation metrics such as Intersection over Union (IoU), Dice coefficient, precision, and recall are preferred over simple accuracy, as they better reflect segmentation performance under imbalance.

Sample Illustration:

Figure 1 presents an example from the dataset. The input image depicts a castle located near a body of water, surrounded by a visually rich background including vegetation and architectural elements. The corresponding ground truth mask highlights only the castle as the salient object, represented as a white silhouette against a black background. The goal of the model is to correctly identify and segment this foreground object while ignoring the surrounding background.

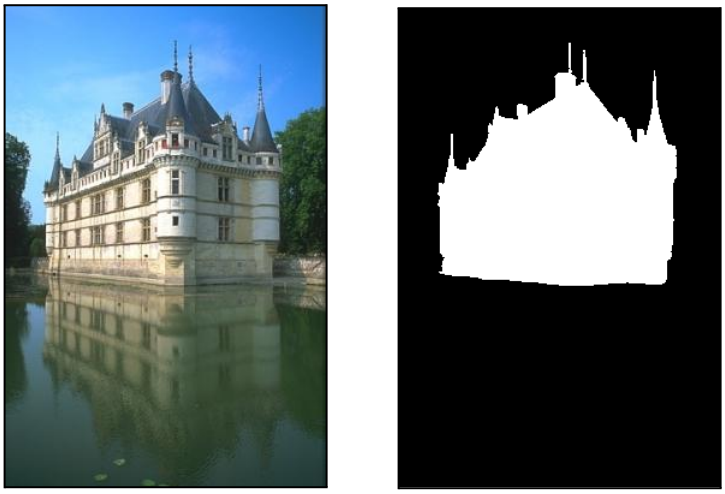

Fig. 1. Sample illustration of an image in the ECSSD dataset [2].

2.2 Dataset Preprocessing

The dataset is divided into three disjoint subsets to enable training, validation, and unbiased evaluation. A **70%-15%-15%** split is used, corresponding to 700 training images, 150 validation images, and 150 test images. The split is performed randomly with a fixed seed to ensure reproducibility. The validation set is used for model selection and will be used for hyperparameter tuning in later stages, while the test set is reserved strictly for final performance evaluation.

Image and Mask Preprocessing:

To ensure consistency across different models, all images and masks are resized to a fixed resolution. Two configurations are used depending on the model architecture:

- **224 x 224** for pretrained Vision Transformer-based models (CLIP and DINO)
- **256 x 256** for CNN-based models (Autoencoder and U-Net)

For image processing, images are resized using bilinear interpolation to preserve visual quality. Pixel values are converted to tensors and scaled to range [0, 1]. For pretrained encoders, images are additionally normalized using model-specific mean and standard deviation values:

- **DINO:** mean = [0.485, 0.456, 0.406] std = [0.229, 0.224, 0.225]
- **CLIP:** mean = [0.48145466, 0.4578275, 0.40821073] std = [0.26862954, 0.26130258, 0.27577711]

For masks preprocessing, masks are resized using nearest-neighbor interpolation to avoid introducing intermediate label values. They are then converted to tensors and binarized using a threshold of **0.5**, ensuring that the segmentation task remains strictly binary.

Data Loading Pipeline:

Image-mask pairs are constructed by matching filenames between the image and mask directories. The dataset loader reads each pair, applies the corresponding transformations, and returns aligned image and mask tensors. Data is loaded in batches using PyTorch DataLoaders, with shuffling enabled for the training set and disabled for validation and test sets. Additionally, pinned memory is used when a GPU is available to improve data transfer efficiency. At this stage of the project, data augmentation techniques have not yet been implemented and are planned for future work.

Key Insights from Preprocessing:

Several important observations arise from the preprocessing stage. First, resizing images to lower resolutions reduces computational cost but may lead to loss of fine boundary details, which can negatively affect segmentation performance. Second, pretrained models such as CLIP and DINO are highly sensitive to input normalization; using incorrect normalization significantly degrades performance. Third, nearest-neighbor interpolation is essential for preserving binary mask integrity during resizing. Finally, preprocessing does not address the inherent pixel-level class imbalance in the dataset, making it necessary to rely on evaluation metrics.

3. Coding Environment

The project is implemented in Python using the PyTorch framework, which provides flexible modules for building and training neural networks. The Torchvision library is used for common image transformations such as resizing, normalization, and tensor conversion. For pretrained vision encoders, the timm (PyTorch Image Models) library is utilized to access state-of-the-art Vision Transformer models.

Specifically, the DINO model is implemented using the **vit_small_patch16_224.dino** [3] architecture, while the CLIP model uses the **vit_base_patch16_224.openai** [4] architecture. These pretrained models are loaded through the timm library, which provides weights trained on large-scale datasets.

Additional libraries are used to support various components of the pipeline. NumPy is employed for numerical operations, while Matplotlib is used for visualizing training curves and precision-recall plots. The scikit-learn library is used for computing precision-recall curves and the area under the curve (AUC) metric. Dataset splitting is also performed using utilities from scikit-learn to ensure reproducibility.

All models are trained in Google Colab using NVIDIA GPU acceleration (CUDA). Models are executed in a notebook that clones the project repository from Github, installs the required dependencies, runs the corresponding training script, and stores or downloads the resulting outputs, including trained model checkpoints and evaluation plots.

4. Details of the Trained Models

In this project, four different segmentation models are implemented and trained to compare architectures learned entirely from scratch with architectures that leverage pretrained visual representations. The selected models are a (i) CNN-based autoencoder, (ii) U-Net, (iii) DINO-based segmenter, and (iv) CLIP-based segmenter. All models take an RGB image as input and produce a single-channel output mask of logits, which is later converted into a binary segmentation map using a sigmoid function and thresholding.

4.1 CNN-Based Autoencoder

The CNN-based autoencoder [5] is used as the simplest baseline model. It follows a standard encoder-decoder architecture and does not include any skip connections. The purpose of this model is to provide a reference point for evaluating improvements from more advanced architectures.

The encoder applies three convolutional blocks. The first block maps the input image from 3 to 16 channels using a 3x3 convolution (with paddings to preserve dimension), followed by a ReLU activation and 2x2 max-pooling, reducing spatial resolution from 256x256 to 128x128. The second block maps 16 to 32 channels and reduces resolution to 64x64, while the third maps 32 to 64 channels, producing a bottleneck feature map of size 64 channels at 32x32 resolution.

The decoder reconstructs the segmentation mask using three transposed convolution layers, each followed by a ReLU activation. These progressively upsample the feature maps:

- 32 x 32 → 64 x 64 (64 → 32 channels)
- 64 x 64 → 128 x 128 (32 → 16 channels)
- 128 x 128 → 256 x 256 (16 → 8 channels)

A final 1x1 convolution produces a single-channel output of raw logits. Because the model is optimized using BCEWithLogitsLoss, no internal sigmoid activation is applied at the final layer. Due to aggressive downsampling and the lack of skip connections, this model loses spatial detail and is expected to produce coarse segmentation results.

4.2 U-Net

The U-Net [6] model is a more advanced encoder–decoder architecture designed for dense prediction tasks like segmentation. It incorporates skip connections between the encoder and decoder layers, allowing the direct preservation of high-resolution spatial details.

The encoder consists of four stages with feature dimensions of 64, 128, 256, and 512. Each stage is implemented using a DoubleConv block (two consecutive 3x3 convolutions with batch normalization and a ReLU activation). Each stage is followed by a 2x2 max-pooling operation, progressively reducing the spatial resolution by half.

A bottleneck layer then expands the feature depth from 512 to 1024 channels at the lowest resolution, capturing global semantic context.

The decoder mirrors the encoder. Each stage upsamples the feature map using a 2x2 transposed convolution, concatenates it with the corresponding encoder feature map via skip connections, and refines the combined result using another DoubleConv block. A final 1x1 convolution produces the single-channel output of raw logits.

U-Net is particularly suitable for ECSSD because it combines deep semantic understanding (from the bottleneck layers) with precise localization (from the skip connections).

4.3 DINO-Based Segmentation Model

The DINO-based model [7] uses a pretrained Vision Transformer (ViT-Small Patch16) trained via self-supervised learning on ImageNet-1K, loaded via the timm library. The encoder is used as a frozen feature extractor, meaning its parameters are strictly locked during training to preserve the learned representations.

The input image (224x224) is divided into 16x16 patches, resulting in 14x14=196 tokens. After removing the optional [CLS] token, these tokens are reshaped into a 2D spatial feature map of size [B, C, 14, 14], where C is the embedding dimension of the ViT.

The decoder maps this low-resolution semantic representation back to a full-resolution mask. It consists of:

- A 3x3 convolution mapping the embedding dimension to 256 channels, followed by batch normalization and ReLU activation.
- Four transposed convolutions to upsample the feature map back to 224x224. At each of the four steps (14→28→56→112→224), the spatial resolution doubles while the channel depth is halved (256→128→64→32→16).
- A final 1x1 convolution to produce the single-channel output of raw logits.

DINO provides strong semantic representations due to large-scale self-supervised pretraining on ImageNet, making it well-suited for small datasets like ECSSD. However, its low-resolution feature map (14x14) limits fine boundary precision compared to architectures that utilize high-resolution skip connections.

4.4 CLIP-Based Segmentation Model

The CLIP-based model [8] uses a pretrained ViT-Base Patch16 encoder (vit_base_patch16_clip_224.openai), trained on large-scale image–text pairs using contrastive learning. Like DINO, the encoder is strictly frozen to act solely as a feature extractor without degrading its pretrained language-vision alignment during training.

The 224x224 input image is processed into 16x16 patches. After stripping the [CLS] token to reduce the sequence length from 197 to 196, the remaining patch tokens are reshaped into a 2D spatial feature map of size [B, C, 14, 14], where C represents the CLIP embedding dimension. To ensure a fair and controlled comparison, this feature map is passed through a decoder architecture that is perfectly identical to the DINO model.

CLIP's advantage lies in its ability to capture high-level semantic meaning, which is important in ECSSD where saliency depends on object significance rather than only visual contrast. However, similar to DINO, its patch-based structure and low-resolution bottleneck (14x14) limit precise spatial localization at the boundaries.

4.5 Hyperparameters and Their Effects

Several hyperparameters influence model performance:

- **Learning Rate:** Determines the step size during gradient descent, directly affecting convergence speed and overall training stability.
- **Batch Size:** Impacts the variance of gradient estimation and dictates memory utilization. Balancing this parameter is crucial for providing enough stochastic noise to escape local minima while maintaining stable weight updates.
- **Number of Epochs:** Dictates the total training duration. Proper monitoring of the validation metrics across epochs is required to migrate the risk of overfitting, especially on smaller datasets.
- **Input Resolution:** Represents a critical trade off between computational efficiency and spatial accuracy. While lower resolutions process faster, they inherently cap the precision of boundary delineations.
- **Binarization Threshold:** The threshold applied to the final sigmoid probabilities dictates the precision-recall tradeoff.

Furthermore, for the pretrained architectures, freezing the encoder significantly reduces computational cost and prevents the forgetting of learned representations. Finally, applying the correct statistical normalization (ImageNet or OpenAI pretraining statistics) to the input tensors is critical for the frozen feature extractors to function correctly.

5. Experimental Setup

All models are trained under a consistent setup to ensure a fair comparison. The main training configurations are summarized in Table 1.

Table 1: Training Configuration

Hyperparameter	CNN / U-Net	DINO / CLIP
Optimizer	Adam	Adam
Learning Rate	1e-3	1e-3
Loss Function	BCEWithLogitsLoss	BCEWithLogitsLoss
Epochs	10	10
Batch Size	16	16
Input Size	256 x 256	224 x 224
Encoder Frozen	No	Yes

All models are evaluated on the same held-out test set of 150 images to ensure fairness. A fixed random seed (**42**) is used throughout training and data splitting to ensure reproducibility.

For the pretrained models (DINO and CLIP), only the decoder parameters are optimized. This is achieved by setting "requires_grad=False" for all encoder parameters and passing only the decoder parameters to the optimizer.

The Adam [9] optimizer is used for all experiments. Adam computes individual adaptive learning rates for different parameters by tracking both the moving average of the gradients and the squared gradients, which ensures stable and efficient convergence. For the objective function, BCEWithLogitsLoss [10] is used. This function combines a Sigmoid layer with Binary Cross-Entropy in a numerically stable way. It calculates the pixel-by-pixel classification error between the model's raw unnormalized predictions (logits) and the true binary ground truth mask, making it the optimal choice for binary segmentation tasks.

Training Procedure and Model Selection:

During training, each model is evaluated on the validation set after every epoch. The **Dice coefficient** is utilized as the primary selection criterion. The model weights achieving the highest validation Dice score are saved to the "outputs/checkpoints/" directory, preventing the retention of overfitted weights. After training is completed, the best-performing model is reloaded and evaluated on the held-out test set to obtain the final performance results. To compute these metrics, the model's raw output logits are converted into binary segmentation masks by applying a sigmoid activation function followed by a binarization threshold of **0.5**.

Evaluation Metrics:

To comprehensively evaluate segmentation performance, five standard threshold-dependent metrics are used, defined by True Positives (TP), False Positives (FP), and False Negatives (FN):

- **Intersection over Union (IoU):** Measures the absolute spatial overlap between the predicted and ground truth masks.

$$IoU = \frac{TP}{TP + FP + FN} \quad (1)$$

- **Dice Coefficient:** Measures the similarity between the prediction and ground truth, giving slightly more weight to correct positive predictions.

$$Dice = \frac{2TP}{2TP + FP + FN} \quad (2)$$

- **Precision:** Measures the accuracy of positive predictions (how many predicted foreground pixels are correct).

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

- **Recall:** Measures the completeness of the predictions (how many true foreground pixels are successfully detected).

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

- **F-measure:** The harmonic mean of precision and recall.

$$F = \frac{2 . Precision . Recall}{Precision + Recall} \quad (5)$$

In addition to these metrics, a precision–recall (PR) curve is computed by sweeping the decision threshold over the predicted probabilities. The area under the PR curve (PR AUC) is reported as a robust, threshold-independent performance measure.

Experimental Goals:

The primary goal of the experiments is to compare the effectiveness of pretrained, frozen Vision Transformer encoders (DINO and CLIP) against architectures trained from scratch (CNN and U-Net) on a moderately sized ECSSD dataset. In particular, the study investigates whether pretrained representations improve overall segmentation performance, provide better sample efficiency, and offer superior generalization. Moreover, it contrasts the high-level semantic understanding offered by DINO and CLIP against the precise spatial localization achieved by U-Net's skip connections. A secondary goal is to analyze the training dynamics and convergence behavior of each model, including how quickly they learn and how stable their performance remains throughout training.

6. Results

6.1 CNN Results

The CNN autoencoder exhibits underfitting behavior with a strong bias toward the background class, achieving a low Test F-measure of **0.1037**. For the first seven epochs, all validation metrics remained at **0.0000**, indicating that the model converged to predicting only background due to class imbalance. Meaningful segmentation begins only after epoch 8, when some foreground predictions surpass the 0.5 threshold. However, the final recall (**0.0763**) remains very low, showing that the model detects only small portions of the salient object.

Table 2: Final Test Performance (CNN Autoencoder)

Model	Loss	IoU	Dice	Precision	Recall	F-measure	PR AUC
CNN	0.4762	0.0674	0.1037	0.4161	0.0763	0.1037	0.4013

Training and Validation Loss:

The loss curves (Fig. 2) show a steady decrease (Train: 0.6069 → 0.4952), indicating stable optimization. The small gap between training and validation loss suggests no overfitting. Instead, the model is underfitting, as it lacks the capacity to capture complex spatial structures without skip connections.

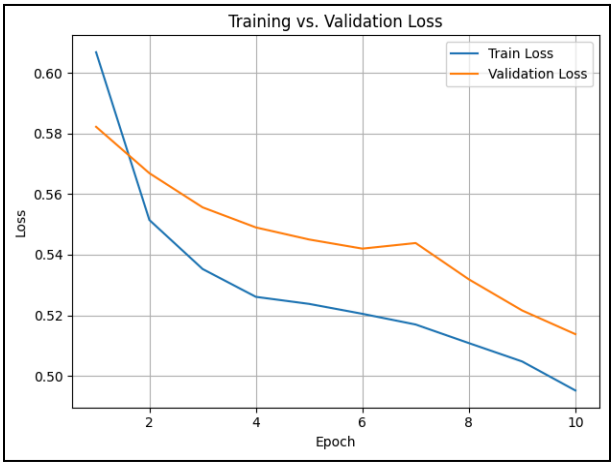

Fig. 2. Training and validation loss curves of CNN.

Precision-Recall Curve:

The PR curve (Fig. 3) yields a low PR AUC of **0.4013**, confirming weak performance. The model maintains moderate precision only at very low recall levels (<10%), but precision drops quickly as recall increases. This indicates that predictions are sparse and fail to capture complete object regions.

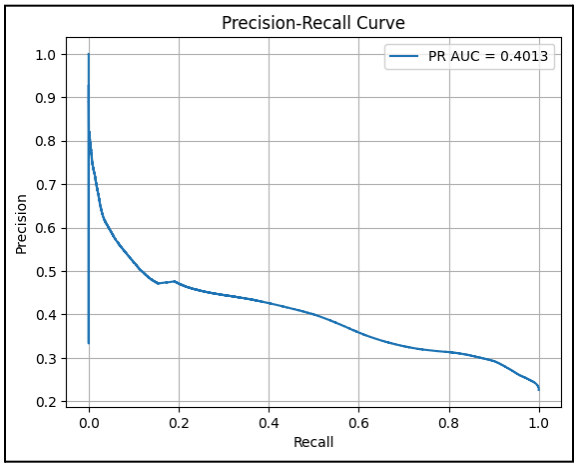

Fig. 3. Precision-Recall curve of CNN (PR AUC = 0.4013).

6.2 U-Net Results

The U-Net architecture shows a significant improvement over the baseline CNN, achieving a Test F-measure of **0.5870**. Its most notable characteristic is very high recall (**0.8543**) with moderate precision (**0.5013**), indicating that it successfully captures most of the salient object. However, the lower precision suggests some over-segmentation, where background regions are incorrectly classified as foreground. This behavior is expected when training a high-capacity model from scratch on a relatively small dataset, where the model captures object structure but struggles to suppress background noise.

Table 3: Final Test Performance (U-Net)

Model	Loss	IoU	Dice	Precision	Recall	F-measure	PR AUC
U-Net	0.4588	0.4370	0.5870	0.5013	0.8543	0.5870	0.6534

Training and Validation Loss:

The training loss decreases smoothly (0.5200 → 0.3646), while validation loss shows noticeable fluctuations, reflecting sensitivity to the small validation set. The best model is obtained at epoch 9 (Dice = **0.6367**), despite a higher validation loss, indicating that BCE loss and Dice optimize different aspects. The model achieves better structural overlap by increasing recall, even at the cost of higher loss.

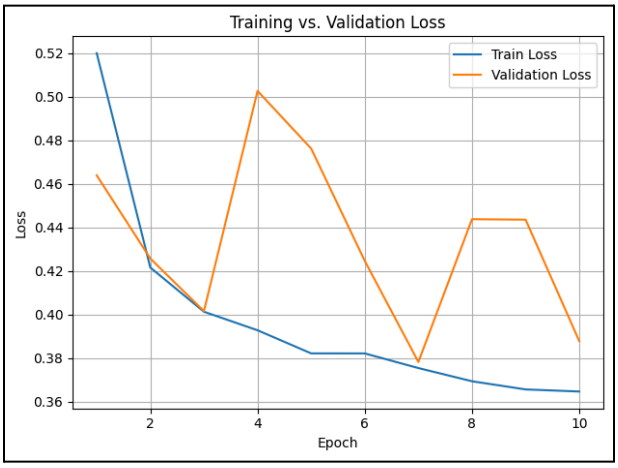

Fig. 4. Training and validation loss curves of U-Net.

Precision-Recall Curve:

The PR curve (Fig. 5) shows strong improvement over the baseline, with PR AUC = **0.6534**. The curve remains stable, maintaining reasonable precision at higher recall levels compared to CNN. This indicates a balanced trade-off, where the model successfully captures most of the object while gradually including background regions.

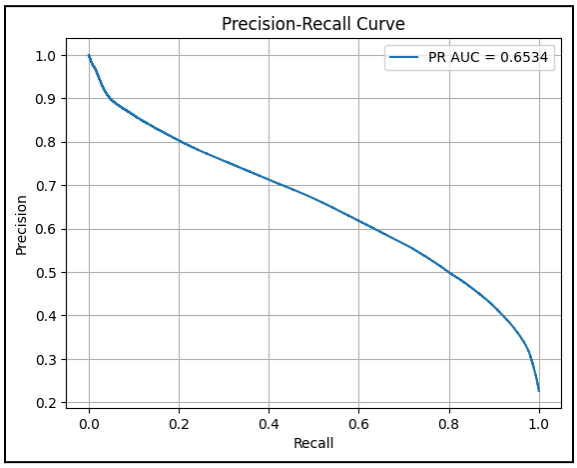

Fig. 5. Precision-Recall curve of U-Net (PR AUC = 0.6534).

6.3 DINO Results

The DINO-based model shows a major improvement over models trained from scratch, achieving a Test F-measure of **0.8721**. Due to self-supervised pretraining, the encoder provides strong semantic and spatial representations, allowing the model to reach a Validation Dice of **0.8608** in the first epoch. Final results show both high precision (**0.8627**) and high recall (**0.8999**), indicating accurate and well-balanced segmentation with minimal false positives.

Table 4: Final Test Performance (DINO)

Model	Loss	IoU	Dice	Precision	Recall	F-measure	PR AUC
DINO	0.1620	0.7864	0.8721	0.8627	0.8999	0.8721	0.9578

Training and Validation Loss:

The loss curves (Fig. 6) show smooth and stable convergence, unlike the fluctuations observed in U-Net. The best model is obtained at epoch 5 (Val Dice: **0.8864**). After this point, training loss continues to decrease while validation loss begins to rise slightly, indicating mild overfitting of the trainable decoder. The checkpointing strategy successfully captures the optimal model before overfitting.

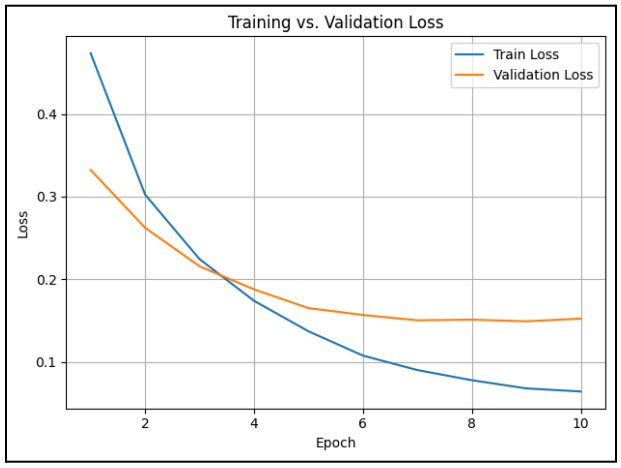

Fig. 6. Training and validation loss curves of DINO.

Precision-Recall Curve:

The PR curve (Fig. 7) is exceptionally strong, with a PR AUC of **0.9578**. The curve remains near the top-right region, maintaining almost perfect precision up to high recall levels (~0.8) before gradually decreasing. This indicates that the model can accurately capture most of the salient object while keeping false positives minimal, demonstrating the effectiveness of pretrained DINO features.

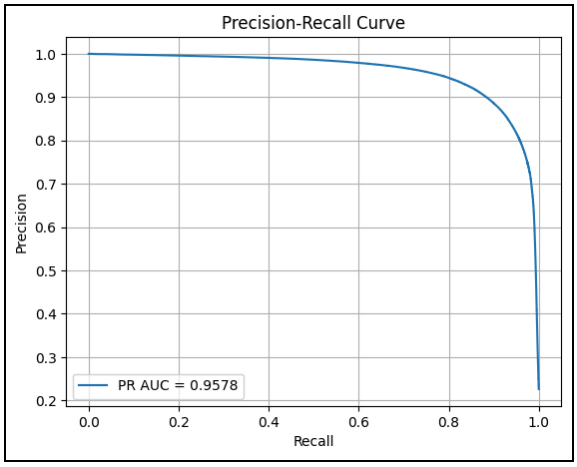

Fig. 7. Precision-Recall curve of DINO (PR AUC = 0.9578).

6.4 CLIP Results

The CLIP-based model achieves strong performance with a Test F-measure of **0.8430**, significantly outperforming models trained from scratch. However, it slightly trails DINO (**0.8430** vs. **0.8721**), reflecting differences in pretraining. CLIP's image-text contrastive training provides strong semantic understanding, enabling accurate object identification, but slightly less precise in fine boundary delineation compared to DINO.

Table 5: Final Test Performance (CLIP)

Model	Loss	IoU	Dice	Precision	Recall	F-measure	PR AUC
CLIP	0.1677	0.7458	0.8430	0.8415	0.8778	0.8430	0.9367

Training and Validation Loss:

The loss curves (Fig. 8) show stable convergence, with training loss decreasing from 0.5012 to 0.0830. Validation loss follows a smooth decline before plateauing, indicating effective transfer learning. The best model is obtained at epoch 8 (Val Dice: **0.8481**). After this point, a widening gap between training and validation loss suggests the onset of overfitting of the trainable decoder, which is mitigated by checkpointing.

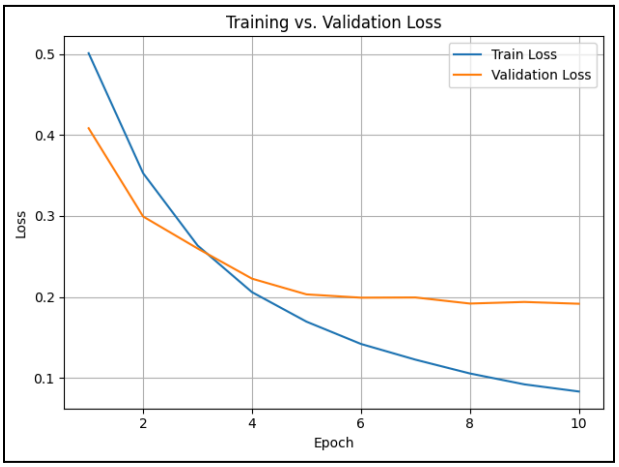

Fig. 8. Training and validation loss curves of CLIP.

Precision-Recall Curve:

The PR curve (Fig. 9) is strong, with PR AUC = **0.9367**, maintaining high precision (>0.90) up to relatively high recall (~0.75). Compared to DINO, the curve drops slightly faster, indicating reduced confidence near object boundaries. This confirms that CLIP captures the core semantic regions well but is less precise at finer spatial details.

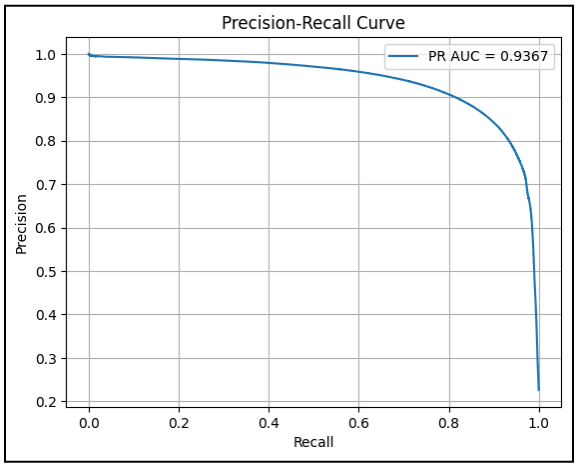

Fig. 9. Precision-Recall curve of CLIP (PR AUC = 0.9367).

7. Future Work

First, hyperparameter tuning has not yet been performed. A systematic exploration of learning rates, batch sizes, and training duration could further improve performance, particularly for models trained from scratch such as CNN and U-Net.

Second, data augmentation techniques (e.g., horizontal flipping, random cropping, and color jittering) have not yet been incorporated. These are expected to improve generalization and reduce overfitting, especially given the relatively small size of the ECSSD dataset.

Third, sample efficiency will be investigated by training the models on varying fractions of the training dataset (e.g., 25%, 50%, and 100%). This will help evaluate how well pretrained architectures maintain performance under limited data compared to models trained from scratch.

Finally, the proposed **DINO + CLIP feature fusion model**, described in the project proposal, has not yet been implemented. Combining complementary semantic representations from both encoders may lead to further improvements in segmentation performance.

8. Workload Distribution

- **Yunus Günay**: Codebase development, implementation of model architectures, and report writing.
- **Daib Malik**: Experiment execution, results analysis, contribution to model implementations, and report writing.
- **Muhammad Musa Qureshi:** Dataset acquisition, contribution to CNN implementation, and report writing.
- **Ali Sher Mir:** Generation of visualizations, contribution to U-Net implementation, and report writing.

9. References

[1] Shi, J., Yan, Q., Xu, L., & Jia, J. (2015). Hierarchical Image Saliency Detection on Extended CSSD. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI).*

[2] "Extended Complex Scene Saliency Dataset (ECSSD)." The Chinese University of Hong Kong. [Online]. Available: https://www.cse.cuhk.edu.hk/leojia/projects/hsaliency/dataset.html [Accessed: Apr. 5, 2026].

[3] timm, "vit_small_patch16_224.dino," *Hugging Face*, 2025. [Online]. Available: https://huggingface.co/timm/vit_small_patch16_224.dino . [Accessed: Apr. 5, 2026].

[4] timm, "vit_base_patch16_clip_224.openai," *Hugging Face Model Repository*. [Online]. Available: https://huggingface.co/timm/vit_base_patch16_clip_224.openai . [Accessed: Apr. 5, 2026].

[5] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, Dec. 2015. doi: 10.48550/arXiv.1511.08458.

[6] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *arXiv preprint arXiv:1505.04597*, May 2015. doi: 10.48550/arXiv.1505.04597.

[7] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," *arXiv preprint arXiv:2104.14294*, 2021.

[8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," *arXiv preprint arXiv:2103.00020*, 2021.

[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[10] "BCEWithLogitsLoss — PyTorch documentation." *PyTorch*. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html [Accessed: Apr. 5, 2026].